

Wyczerpujący przewodnik tworzenia projektów Machine Learning

Od surowych danych CSV do dostrojonego modelu — na zbiorze
California Housing

Pipeline · Przeciwdziałanie overfittingowi · Najtrudniejsze kwestie

Oparty o notatnik `P2Lab10_housing_essential_ml.ipynb`. Wyjaśnia nie tylko CO robi kod, ale
DLACZEGO tak, JAK zbudowane są funkcje i GDZIE czyhają pułapki.

Spis treści

1. Cykl życia projektu ML
2. Wczytanie i eksploracja danych
3. Podział danych — losowanie warstwowe
4. Inżynieria cech i korelacje

CZĘŚĆ I — PIPELINE (serce projektu)

5. Pipeline krok po kroku
6. Własny transformer (BaseEstimator + TransformerMixin)
7. Łączenie pipeline'ów: ColumnTransformer

CZĘŚĆ II — PRZECIWDZIAŁANIE OVERFITTINGOWI

8. Modele i diagnoza overfittingu
9. Walidacja krzyżowa (Cross-Validation)
10. Strojenie hiperparametrów (Grid / Randomized Search)
11. Ostateczna ewaluacja — moment prawdy
12. Checklista i najczęstsze błędy

1. Cykl życia projektu ML

Każdy projekt ML — niezależnie od dziedziny — przebiega według tego samego żelaznego cyklu. Kolejność kroków NIE jest przypadkowa.

```
Dane -> PODZIAŁ (sejf testowy) -> Eksploracja (tylko train) -> Pipeline ->
-> Trening wielu modeli -> Walidacja krzyżowa -> Strojenie -> OTWARCIE SEJFU (test)
```

Data Snooping Bias / Data Leakage: Jeżeli w jakikolwiek sposób użyjesz danych testowych przy budowie modelu (nawet do policzenia mediany do uzupełnienia braków!), model „podejrzy” odpowiedzi. Ocena na końcu będzie fałszywie świetna, a w produkcji model zawiedzie. Dlatego podział na train/test robimy PRZED eksploracją i czyszczeniem, a zbiór testowy zamykamy w „sejfie”.

2. Wczytanie i eksploracja danych

`pd.read_csv()` wczytuje plik CSV do obiektu `DataFrame` (tabela z indeksami) w RAM.

```
import pandas as pd
import numpy as np

def load_housing_data(housing_path="content:housing.csv"):
    return pd.read_csv(housing_path)

housing = load_housing_data()
```

Cztery narzędzia diagnostyczne

Metoda	Co pokazuje	Czego szukasz
<code>housing.info()</code>	typy kolumn, liczba non-null	braki danych i kolumny tekstowe (object)
<code>housing.describe()</code>	mean, std, kwartyle	skośność (mean != mediana), skala
<code>...value_counts()</code>	liczność każdej kategorii	proporcje klas
<code>housing.hist(bins=50)</code>	histogramy rozkładów	wartości ucięte (capped), różne skale

Jak czytać `info()`

```
total_bedrooms    20433 non-null float64    # ALARM: 207 braków (NaN) -> potrzebny Imputer
ocean_proximity   20640 non-null object      # tekst -> potrzebny OneHotEncoder
```

Co wychwycić na histogramach

- Wartości ucięte (capped): pionowy słupek przy `median_house_value = 500 000` oznacza, że wszystkie droższe domy wpisano jako 500k. Model nauczy się fałszywego sufitu cen.
- Różne skale: jedne cechy to dziesiątki, inne dziesiątki tysięcy -> konieczny Feature Scaling.

3. Podział danych — losowanie warstwowe

Losowy `train_test_split` może przypadkiem dać niereprezentatywny zbiór testowy. Skoro `median_income` to najsilniejszy predyktor ceny, chcemy by proporcje grup dochodowych były identyczne w train i test. To Stratified Sampling (losowanie warstwowe).

Krok 1: stworzenie kategorii dochodu (warstwy)

```
housing["income_cat"] = np.ceil(housing["median_income"] / 1.5) # ~5 koszyków
housing["income_cat"] = housing["income_cat"].clip(upper=5)      # wszystko >5 -> 5
```

- / 1.5 — zmniejsza liczbę kategorii (zagęszcza koszyki).
- np.ceil() — „sufit”, zaokrąglenie w górę -> dyskretne klasy 1.0, 2.0, 3.0...
- .clip(upper=5) — przycina górę: rzadkie wysokie dochody scalamy w jeden koszyk 5.

Krok 2: podział z zachowaniem proporcji

```
from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_index, test_index in split.split(housing, housing["income_cat"]):
    strat_train_set = housing.loc[train_index] # .loc = dostęp po etykiecie indeksu
    strat_test_set = housing.loc[test_index]
```

Najtrudniejsze: Subtelność: .split() to GENERATOR zwracający tablice INDEKSÓW (nie gotowe dane). Pierwszy argument to dane, drugi — kolumna wg której zachować proporcje. Stąd pętla for z .loc[...].

Krok 3: sprzątanie i izolacja

```
for set_ in (strat_train_set, strat_test_set):
    set_.drop(["income_cat"], axis=1, inplace=True) # axis=1 = kolumna; inplace = nadpisz

housing = strat_train_set.copy() # kopia: zbiór testowy jest „święty”
```

Sejf zamknięty: Od tej chwili strat_test_set nie istnieje. Nie analizujemy go, nie wizualizujemy, nie liczymy z niego statystyk. Wraca dopiero w sekcji 11.

4. Inżynieria cech i korelacje

Surowa cecha bywa bezużyteczna bez kontekstu. „Liczba pokoi w całej dzielnicy” nic nie mówi — ale „pokoje na gospodarstwo” już tak.

```
housing["rooms_per_household"] = housing['total_rooms'] / housing['households']
housing["bedrooms_per_room"] = housing['total_bedrooms'] / housing['total_rooms']
housing["population_per_household"] = housing['population'] / housing['households']
```

Korelacja Pearsona

```
corr_mat = housing_num.corr()
corr_mat["median_house_value"].sort_values(ascending=False)
```

- +0.68 (median_income) -> silny związek dodatni: rosną dochody -> rosną ceny.
- bliski 0 -> brak związku LINIOWEGO (nieliniowy może istnieć!).
- -1 -> ruch przeciwny.

Nowa cecha bedrooms_per_room ma zwykle silniejszą (ujemną) korelację z ceną niż surowe total_bedrooms — to dowód, że inżynieria cech działa.

Oddzielenie etykiet (X i y)

```
housing      = strat_train_set.drop("median_house_value", axis=1) # X – cechy
housing_labels = strat_train_set["median_house_value"].copy()    # y – cel
```

CZĘŚĆ I

PIPELINE — serce projektu

5. Pipeline krok po kroku

Modele to maszyny matematyczne: jedzą wektory liczb. Tekst, puste komórki czy cechy różnej skali je psują. Dlatego cały preprocessing zamykamy w Pipeline.

Dlaczego Pipeline, a nie ręczne czyszczenie? Trzy twarde powody

- Powtarzalność train->produkcja: w produkcji przetwarzasz pojedyncze próbki; pipeline wykonuje dokładnie ten sam ciąg operacji z zapamiętanymi parametrami.
- Brak data leakage: pipeline uczy parametry (medianę, skalę) tylko na treningu.
- Strojenie: cały pipeline (łącznie z czyszczeniem) można optymalizować w Grid Search.

Święta Trójca metod scikit-learn

Metoda	Rola	Zwraca
fit(X)	uczy się z danych (mediana, std, lista kategorii)	self
transform(X)	stosuje wyuczone parametry, przekształca dane	nową tablicę
fit_transform(X)	uczy się i od razu przekształca (szybsze)	nową tablicę

NAJWAŻNIEJSZE ROZRÓŻNIENIE W CAŁYM ML: `fit_transform()` — TYLKO na zbiorze treningowym (uczy + przekształca). `transform()` — na teście i w produkcji (przekształca STARYMI parametrami z treningu). Użycie `fit_transform()` na teście = data leakage = katastrofa (patrz sekcja 11).

Pipeline numeryczny — trzy ogniwa

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")), # 1. uzupełnij braki
    ('attribs_adder', CombinedAttributesAdder()), # 2. dodaj nowe cechy
    ('std_scaler', StandardScaler()), # 3. wyskaluj
])
```

Pipeline przyjmuje listę krotek ("nazwa", obiekt). Wykonuje `fit_transform` na pierwszym ogniwie, wynik podaje do drugiego itd. Na ostatnim estymatorze (jeśli to model) woła `fit`.

1. SimpleImputer(strategy="median")

Uzupełnia NaN. W `fit()` liczy i zapamiętuje medianę każdej kolumny; w `transform()` wstawia ją w miejsce braków. Dlaczego mediana, nie średnia? Mediana jest odporna na anomalie (skrajnie drogie domy nie wypaczą jej tak jak średniej).

2. CombinedAttributesAdder()

Nasz własny transformer (sekcja 6).

3. StandardScaler() — standaryzacja (z-score)

$$X_{scaled} = (X - \mu) / \sigma$$

Po niej każda cecha ma średnią ≈ 0 i odchylenie ≈ 1 (wartości od ok. -2.5 do 2.5). Bez tego cechy w milionach „przygłuszają” cechy w dziesiątkach przy liczeniu odległości/gradientu.

Wskazówka: Mit: „skalowanie nie jest potrzebne dla drzew”. Prawda — drzewa i lasy są niewrażliwe na skalę. Ale skalowanie nie szkodzi, a ten sam pipeline obsługuje też regresję liniową, więc trzymamy je dla spójności.

Pipeline kategoryjny — OneHotEncoder

```
from sklearn.preprocessing import OneHotEncoder

cat_pipeline = Pipeline([
    ('one_hot', OneHotEncoder()),
])
```

Dlaczego nie zwykłe liczby INLAND=1, NEAR BAY=2? Bo model uzna, że $2 > 1$, czyli że „NEAR BAY jest większe/lepsze od INLAND” — to fałszywa relacja porządku.

One-Hot tworzy osobną kolumnę 0/1 dla każdej kategorii.

```
ocean_proximity = "INLAND"    -> [0, 1, 0, 0, 0]
ocean_proximity = "NEAR BAY" -> [0, 0, 0, 1, 0]
```

Sparse matrix: Wynikiem jest macierz rzadka (sparse matrix) — niemal same zera, przechowywane oszczędnie. To normalne; jeśli potrzebujesz zwykłej tablicy: `OneHotEncoder(sparse_output=False)`.

6. Własny transformer

Gdy chcemy dołożyć cechy wewnątrz pipeline'u (a nie ręcznie), piszemy własną klasę. Musi dziedziczyć po dwóch klasach bazowych.

```
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np

# indeksy kolumn (NumPy gubi nazwy, zostają tylko pozycje!)
rooms_ix, bedrooms_ix, population_ix, household_ix = 3, 4, 5, 6

class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
    def __init__(self, add_bedrooms_per_room=True): # przełącznik = hiperparametr
        self.add_bedrooms_per_room = add_bedrooms_per_room

    def fit(self, X, y=None):
        return self # nic się nie uczy -> zwraca self

    def transform(self, X, y=None):
        rooms_per_household = X[:, rooms_ix] / X[:, household_ix]
        population_per_household = X[:, population_ix] / X[:, household_ix]
        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, bedrooms_ix] / X[:, rooms_ix]
            return np.c_[X, rooms_per_household, population_per_household, bedrooms_per_room]
        return np.c_[X, rooms_per_household, population_per_household]
```

Rozbiór najtrudniejszych elementów

- `BaseEstimator` -> daje za darmo `get_params()` / `set_params()`. Bez tego Grid Search nie umiałby ustawiać hiperparametrów tej klasy.
- `TransformerMixin` -> daje za darmo `fit_transform()` (kombinację `fit` + `transform`).
- `__init__` bez `*args/**kwargs` -> wymóg sklearn: każdy hiperparametr musi być osobnym, nazwanym argumentem przypisanym do `self` bez zmian. `add_bedrooms_per_room` pozwala w Grid Search sprawdzić, czy ta cecha w ogóle pomaga.

- `X[:, rooms_ix]` -> notacja NumPy: ':' = wszystkie wiersze, `rooms_ix` = kolumna nr 3. Dzielenie dwóch wektorów to operacja zwektoryzowana (błyskawiczna, bez pętli).
- `np.c_[...]` -> skleja kolumny obok siebie („dokłada z prawej”). `X` ma `N` kolumn, wynik `N+2` lub `N+3`.

Najtrudniejsze: Dlaczego indeksy liczbowe a nie nazwy? Bo zanim transformer dostanie dane, `SimpleImputer` zamienił `DataFrame` na tablicę NumPy — a NumPy nie ma nazw kolumn, tylko pozycje. Stąd `rooms_ix = 3`.

7. Łączenie pipeline'ów: ColumnTransformer

Mamy dwa pipeline'y (liczbowy i kategoryjny) — trzeba skierować odpowiednie kolumny do odpowiedniego.

```
from sklearn.compose import ColumnTransformer

num_attribs = ['longitude', 'latitude', 'housing_median_age', 'total_rooms',
               'total_bedrooms', 'population', 'households', 'median_income']
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([
    ("num_pipeline", num_pipeline, num_attribs),    # te kolumny -> pipeline liczbowy
    ("cat_pipeline", cat_pipeline, cat_attribs),    # te kolumny -> pipeline OHE
])

housing_prepared = full_pipeline.fit_transform(housing)    # fit_transform TYLKO tu (train)
```

`ColumnTransformer` to „inteligentna brama”: kieruje wskazane kolumny do wskazanego transformera, a wyniki skleja w jedną macierz. Po tej linii `housing_prepared` to czysta, liczbową macierz NumPy gotową do treningu.

CZĘŚĆ II

PRZECIWDZIAŁANIE OVERFITTINGOWI

8. Modele i diagnoza overfittingu

Trenujemy trzy modele od najprostszego do najlepszego i obserwujemy, jak overfitting się ujawnia i jak go zwalczać.

Model 1: Regresja liniowa — UNDERfitting

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

lin_reg = LinearRegression()
lin_reg.fit(housing_prepared, housing_labels)

housing_predictions = lin_reg.predict(housing_prepared)
lin_rmse = np.sqrt(mean_squared_error(housing_labels, housing_predictions))
```

Wynik:
lin_rmse ≈ 68 628 \$ # model myli się średnio o ~70 tys. USD

Model: $\hat{y} = \theta_0 + \theta_1 \cdot x_1 + \dots + \theta_n \cdot x_n$ (hiperpłaszczyzna). fit() dobiera wagi θ minimalizujące sumę kwadratów błędów. RMSE 68 628 \$ to UNDERFITTING — model jest za prosty, by uchwycić nieliniowe zależności. Objaw: duży błąd już na treningu. Leki: mocniejszy model, lepsze cechy, mniej regularyzacji.

Model 2: Drzewo decyzyjne — OVERfitting podręcznikowy

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor()
tree_reg.fit(housing_prepared, housing_labels)

housing_predictions = tree_reg.predict(housing_prepared)
tree_rmse = np.sqrt(mean_squared_error(housing_labels, housing_predictions))
```

Wynik:
tree_rmse ≈ 0.0 \$ (!!!)

OVERFITTING: RMSE = 0 to NIE sukces — to czerwony alarm. Drzewo bez ograniczeń rozrasta się aż „zapamięta na blachę” każdy dom z treningu. Na zbiorze testowym wypadnie fatalnie. Model nauczył się odpowiedzi na pamięć zamiast GENERALIZOWAĆ.

Jak zdiagnozować, skoro na treningu wynik = 0? -> walidacja krzyżowa (sekcja 9) pokaże prawdziwy błąd ~71 tys., gorszy nawet od regresji liniowej. Leki (regularyzacja przez pruning): max_depth, min_samples_leaf, min_samples_split, max_leaf_nodes.

Model 3: Las losowy — kompromis przez Ensemble

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = RandomForestRegressor()
forest_reg.fit(housing_prepared, housing_labels)
```

Wynik:
RMSE train ~18 tys.; w walidacji krzyżowej ~50 tys. — najlepszy z trójki

Random Forest = Ensemble Learning + Bagging (Bootstrap Aggregating): (1) buduje setki drzew, każde na innej losowej próbce danych (bootstrap) i losowym podzbiorze cech; (2) każde drzewo jest słabe i przeuczone inaczej — ich błędy są niezależne; (3) wynik to uśrednienie wszystkich drzew -> szumy i anomalie się znoszą.

Kluczowa intuicja: Dlaczego las redukuje overfitting? Pojedyncze drzewo ma niską stronniczość, ale ogromną wariancję. Uśrednienie wielu zdekorelowanych drzew drastycznie tnie wariancję zachowując niską stronniczość. To esencja walki z overfittingiem przez „mądrość tłumu”.

9. Walidacja krzyżowa

Zbiór testowy wolno użyć tylko raz, na samym końcu. Gdybyśmy zaglądali do niego po każdej zmianie modelu, pośrednio „uczylibyśmy się” na nim -> leakage. Potrzebujemy oceny bez dotykania sejfu.

K-Fold dzieli zbiór treningowy na K części (foldów). W pętli K razy: trenuje na K-1 foldach, ocenia na pozostałym. Zwraca K wyników -> stabilna średnia + odchylenie.

```
from sklearn.model_selection import cross_val_score

lin_scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
                             scoring="neg_mean_squared_error", cv=10)
lin_rmse_scores = np.sqrt(-lin_scores)

def display_scores(scores):
    print("Scores:", scores)
    print("Mean:", scores.mean())
    print("Std:", scores.std())      # niska = stabilny model

display_scores(lin_rmse_scores)
```

Najtrudniejsze: Pułapka znaku. `cross_val_score` działa wg konwencji „im więcej, tym lepiej”. Skoro błąd ma być mały, `sklearn` używa `neg_mean_squared_error` (wartości UJEMNE). Dlatego liczymy `np.sqrt(-scores)` — najpierw negacja, potem pierwiastek.

Co odkrywa CV

Model	RMSE (CV)	Wniosek
Linear	~69 tys.	underfitting
Tree	~71 tys.	overfitting (na treningu było 0!)
Forest	~50 tys.	najlepiej generalizuje -> idzie do strojenia

Detektor fitu: Jeśli $RMSE_{train} \ll RMSE_{CV}$ -> overfitting. Jeśli oba wysokie -> underfitting. To Twój główny detektor.

10. Strojenie hiperparametrów

Parametry (θ) model uczy sam. Hiperparametry (liczba drzew, głębokość) ustawiasz Ty — i wpływają wprost na overfitting/underfitting.

Grid Search — brutalna siła (każdy z każdym)

```

from sklearn.model_selection import GridSearchCV

param_grid = [
    {'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8]},          # 3x4 = 12
    {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3, 4]}, # 2x3 = 6
]

forest_reg = RandomForestRegressor(random_state=42)
grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)
grid_search.fit(housing_prepared, housing_labels) # (12+6) x 5 = 90 treningów!

print(grid_search.best_params_) # najlepsza kombinacja
best_model = grid_search.best_estimator_ # gotowy, dostrojony model

```

Uwaga: param_grid to LISTA słowników. Grid Search testuje każdą kombinację w każdym słowniku, dla każdej robi pełną walidację krzyżową. Liczba treningów rośnie mnożnikowo — łatwo o eksplozję kombinatoryczną.

Randomized Search — inteligentne losowanie

```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

final_pipeline = Pipeline([
    ('regressor', forest_reg),
])

param_distributions = {
    'regressor__n_estimators': randint(low=3, high=200),
    'regressor__max_features': randint(low=2, high=20),
}

rnd_search = RandomizedSearchCV(final_pipeline, param_distributions=param_distributions,
                                n_iter=10, cv=3,
                                scoring='neg_root_mean_squared_error', random_state=42)
rnd_search.fit(housing_prepared, housing_labels)
print(rnd_search.best_params_)

```

Najtrudniejsze: Podwójny underscore __ to jedna z najtrudniejszych składni sklearn. 'regressor__n_estimators' znaczy: wejdź do kroku pipeline'u o nazwie 'regressor' i ustaw jego parametr n_estimators. Schemat: nazwa_kroku__nazwa_parametru. Dla zagnieżdżeń: preprocessing__num_pipeline__std_scaler__with_mean. Jeden błąd w nazwie = ValueError.

Grid vs Randomized: Grid — gdy mało parametrów i chcesz pewności. Randomized — gdy przestrzeń jest wielka i liczy się czas (często znajduje lepsze ustawienia w mniej prób).

11. Ostateczna ewaluacja — moment prawdy

Czas otworzyć sejf. Tylko raz.

```

X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

X_test_prepared = full_pipeline.transform(X_test) # transform, NIGDY fit_transform!
test_pred = best_model.predict(X_test_prepared)

final_rmse = np.sqrt(mean_squared_error(y_test, test_pred))

```

Wynik:
final_rmse ≈ 47 tys. USD

NAJGROŹNIEJSZY BŁĄD CAŁEGO PROJEKTU: Na zbiorze testowym wołasz `full_pipeline.transform(X_test)`, a NIGDY `.fit_transform()`. Gdybyś użył `fit_transform`, pipeline policzyłby nową medianę i nową skalę z danych testowych. To by (1) wpuściło informację z testu do przetwarzania (data leakage) i (2) przesunęło proporcje względem tych, na których uczył się model. Reguła na całe życie: Trening = `fit_transform()`. Test i produkcja = tylko `transform()`.

Podsumowanie wyników

```
Regresja liniowa:      ~68 600 $   (underfitting)
Drzewo (CV):          ~71 000 $   (overfitting)
Las losowy (CV):      ~50 000 $
Las po strojeniu (TEST): ~47 000 $   <- finalny błąd generalizacji
```

Od „prostej linii” 70k zeszliśmy do 47k — przez inżynierię cech, ensemble i strojenie.

Wskazówka: Po finalnej ewaluacji już nie wracasz do strojenia pod ten wynik — inaczej znów zaczynasz uczyć się na teście.

12. Checklista i najczęstsze błędy

Checklista projektu

- Podział train/test PRZED eksploracją (`StratifiedShuffleSplit`).
- Stratyfikacja wg najważniejszej cechy.
- `housing = strat_train_set.copy()` — izolacja testu.
- Cały preprocessing w Pipeline / `ColumnTransformer` (Imputer + scaler + OHE).
- `fit_transform` tylko na train.
- Ocena modeli przez cross-validation, nie na teście.
- Strojenie przez Grid/Randomized Search z cv.
- `transform` (nie `fit_transform`) na teście; test użyty RAZ, na końcu.

TOP 7 najtrudniejszych pułapek

Pułapka	Skutek	Lek
<code>fit_transform</code> na teście	data leakage, zawyżona ocena	tylko <code>transform</code>
Analiza/statystyki z testu	data snooping bias	sejf do końca
RMSE=0 mylone z sukcesem	ukryty overfitting	cross-validation
Kodowanie kategorii liczbami	fałszywy porządek	<code>OneHotEncoder</code>
Brak skalowania	dominacja dużych cech	<code>StandardScaler</code>
Zła nazwa w __	<code>ValueError</code> w strojeniu	<code>krok__parametr</code>
Brak <code>random_state</code>	brak powtarzalności	ustaw wszędzie =42

Szybka diagnoza fitu

RMSE_train niski, RMSE_CV niski -> OK, dobrze
RMSE_train ~ 0, RMSE_CV wysoki -> OVERFITTING -> regularyzacja, więcej danych, ensemble
RMSE_train wysoki, RMSE_CV wysoki -> UNDERFITTING -> mocniejszy model, lepsze cechy